

Linux Processes - Complete Notes with Examples

I. Processes Overview

Definition: Processes are programs running on your machine, managed by the kernel. Each process has a unique Process ID (PID) assigned in creation order.

Basic Process Viewing - `ps` Command

Simple process list:

```
$ ps
PID      TTY      STAT   TIME       CMD
41230    pts/4    Ss      00:00:00    bash
51224    pts/4    R+      00:00:00    ps
```

Field Explanations:

- **PID:** Process ID
- **TTY:** Controlling terminal associated with the process
- **STAT:** Process status code
- **TIME:** Total CPU usage time
- **CMD:** Name of executable/command

Detailed process list (BSD style):

```
$ ps aux
USER      PID  %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1   0.0  0.1  19356  1544 ?        Ss   Jan01   0:01 /sbin/init
pete     1234   0.5  2.3  45678  9876 pts/0    S    10:30   0:05  bash
pete     5678   0.0  0.1   4567   890 pts/0    R+   10:35   0:00  ps aux
```

Additional Field Explanations:

- **USER:** The effective user (whose access is being used)
- **%CPU:** CPU time used divided by time process has been running
- **%MEM:** Ratio of process's resident set size to physical memory

- **VSZ**: Virtual memory usage of entire process
- **RSS**: Resident set size (non-swapped physical memory used)
- **START**: Start time of the process

Real-time Process Monitoring - **top** Command

top

- Shows real-time process information
- Refreshes every 10 seconds by default
- Useful for identifying resource-intensive processes

2. Controlling Terminal (TTY)

Types of Terminals:

- **Regular terminal devices**: Native terminal devices (TTY1, TTY2, etc.)
- **Pseudoterminals**: What you use in graphical environments (PTS)

Examples:

```
# Switch to TTY1 (virtual console)
Ctrl-Alt-F1

# Return to graphical interface
Ctrl-Alt-F7

# View processes with their terminals
$ ps aux | grep bash
pete      1234    0.0  0.1  21234  4567 pts/0    Ss   10:30   0:00  bash
```

Daemon processes: Show **?** in TTY field as they're not bound to a controlling terminal.

3. Process Details

Key Concepts:

- A process is a running program instance
- Multiple processes can run the same program
- Kernel manages all process resources

Example - Multiple instances of same program:

```
# Terminal 1:
$ cat
# (process stays open waiting for input)

# Terminal 2:
$ cat
# (another instance)

# Terminal 3:
$ ps aux | grep cat
pete      1111    0.0  0.1   4567   890 pts/0    S+   10:30   0:00 cat
pete      2222    0.0  0.1   4567   890 pts/1    S+   10:31   0:00 cat
```

4. Process Creation

Fork System Call: Creates a child process from parent process

- Child gets new PID
- Parent becomes PPID (Parent Process ID)
- Child can use `execve` to run different program

Viewing Parent-Child Relationships:

```
$ ps l
F  UID    PID  PPID  PRI  NI   VSZ   RSS WCHAN  STAT TTY      TIME COMMAND
0  1000   1234   1111   20   0  21234  4567 wait    Ss   pts/0    0:00 bash
0  1000   5678   1234   20   0   4567   890 -       R+   pts/0    0:00 ps l
```

Init Process:

- PID 1 (mother of all processes)
- Created at system boot
- Spawns all other processes
- Cannot be terminated except at shutdown

5. Process Termination

Normal Termination

- Uses `_exit` system call
- Returns termination status (0 = success)
- Parent uses `wait` system call to collect child's status

Orphan Processes

Definition: Child processes whose parent died first

```
# If parent process dies, orphans are adopted by init (PID 1)
$ ps -ef | grep init
root      1      0  0 Jan01 ?          00:00:01 /sbin/init
```

Zombie Processes

Definition: Terminated child processes waiting for parent to collect their status

```
$ ps aux | grep defunct
pete      1234    0.0  0.0      0      0 ?          Z    10:30    0:00
[process_name] <defunct>
```

6. Signals

Definition: Notifications to processes that something has happened

Common Signals

- **SIGHUP (HUP, 1):** Hangup
- **SIGINT (INT, 2):** Interrupt (Ctrl-C)
- **SIGKILL (KILL, 9):** Kill (unblockable)
- **SIGSEGV (SEGV, 11):** Segmentation fault
- **SIGTERM (TERM, 15):** Software termination
- **SIGSTOP (STOP):** Stop/suspend

Examples:

```
# Interrupt a running process
Ctrl-C

# Suspend a process
Ctrl-Z
```

7. Kill Command - Process Termination

Basic Usage:

```
# Send SIGTERM (default)
$ kill 12445

# Send specific signal
$ kill -9 12445      # SIGKILL
$ kill -15 12445     # SIGTERM
$ kill -HUP 12445    # SIGHUP
```

Signal Differences:

- **SIGHUP**: Sent when terminal closes
- **SIGINT**: Graceful interrupt (Ctrl-C)
- **SIGTERM**: Graceful termination with cleanup
- **SIGKILL**: Force kill, no cleanup
- **SIGSTOP**: Suspend process

8. Process Priority - Niceness

Concept: Influence CPU scheduling priority

- High niceness = lower priority (nice to other processes)
- Low/negative niceness = higher priority

Viewing Niceness:

```
$ top
# Look at NI column
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1234	pete	20	0	21234	4567	1234	S	0.0	0.1	0:00.05	bash
5678	pete	20	5	45678	9876	2345	S	0.0	0.2	0:00.10	process

Setting Priority:

```
# Set priority for new process
$ nice -n 5 apt upgrade

# Change priority of existing process
$ renice 10 -p 3245
```

9. Process States

Common State Codes:

- **R**: Running or runnable
- **S**: Interruptible sleep
- **D**: Uninterruptible sleep
- **Z**: Zombie
- **T**: Stopped

Example:

```
$ ps aux
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
pete	1234	0.0	0.1	21234	4567	pts/0	Ss	10:30	0:00	bash
pete	5678	0.0	0.1	4567	890	pts/0	R+	10:35	0:00	ps aux
pete	9999	0.0	0.1	1234	567	pts/0	T	10:20	0:00	vim

10. /proc Filesystem

Concept: Everything in Linux is a file, including processes

Examples:

```
# List all process directories
$ ls /proc
1 2 3 1234 5678 cmdline cpuinfo meminfo version

# View detailed process information
$ cat /proc/1234/status
Name:  bash
State: S (sleeping)
Pid:   1234
PPid:  1111
TracerPid: 0
Uid:   1000    1000    1000    1000
Gid:   1000    1000    1000    1000

# View process command line
$ cat /proc/1234/cmdline
bash

# View process memory maps
$ cat /proc/1234/maps
```

11. Job Control

Background Jobs

Send job to background:

```
$ sleep 1000 &  
[1] 1234  
  
$ sleep 1001 &  
[2] 5678  
  
$ sleep 1002 &  
[3] 9999
```

View background jobs:

```
$ jobs  
[1]      Running      sleep 1000 &  
[2]-    Running      sleep 1001 &  
[3]+    Running      sleep 1002 &
```

Legend:

- `+`: Most recent background job
- `-`: Second most recent background job

Suspend and Background Existing Job

```
$ sleep 1003  
^Z  
[4]+    Stopped      sleep 1003  
  
$ bg  
[4]+    sleep 1003 &  
  
$ jobs  
[1]      Running      sleep 1000 &  
[2]      Running      sleep 1001 &  
[3]-    Running      sleep 1002 &  
[4]+    Running      sleep 1003 &
```

Foreground Jobs

Bring job to foreground:

```
# Bring most recent background job to foreground
$ fg
sleep 1002

# Bring specific job to foreground
$ fg %1
sleep 1000
```

Kill Background Jobs

```
# Kill specific background job
$ kill %1
[1]+  Terminated                  sleep 1000

# Kill all background jobs
$ kill %1 %2 %3
```

Summary Commands Quick Reference

```
# Process viewing
ps                # Simple process list
ps aux           # Detailed process list
ps l             # Long format with PPID
top              # Real-time process monitor

# Process control
kill PID          # Send SIGTERM
kill -9 PID       # Send SIGKILL
kill -15 PID      # Send SIGTERM
killall process_name # Kill all instances

# Job control
command &        # Run in background
jobs             # List background jobs
fg %job_id        # Bring to foreground
bg %job_id        # Send to background
Ctrl-Z           # Suspend current job
Ctrl-C           # Interrupt current job

# Priority
nice -n value cmd # Set priority for new process
renice value -p PID # Change priority of existing process
```



```
# Process information
```

```
/proc/PID/status      # Detailed process info
```

```
/proc/PID/cmdline     # Command line used
```

```
/proc/PID/maps        # Memory maps
```